Design and Implementation of an Autonomous UAV System for Exploration, Mapping, and Object Localization in Subterranean Environments

Leo Heller, Leon Kitzbichler, Brian Kristandra, Dominik Thimm, Sebastian Waeger
TUM School of Engineering and Design, Technical University of Munich, Germany
{leo.heller, leon.kitzbichler, brian.kristandra, dominik.thimm, sebastian.waeger}@tum.de

***Abstract*—Autonomous exploration of subterranean environments poses significant challenges due to limited visibility and unknown structure. This paper describes the design and implementation of an autonomous unmanned aerial vehicle (UAV) system for exploration, mapping, and object localization in subterranean environments. The system is developed using ROS2 and a Unity-based simulation framework. A modular software architecture integrates a perception pipeline that converts depth images into point clouds and a voxel-based map representation, as well as path and trajectory planning modules that enable collision-free navigation. The localization of objects is achieved through the utilization of semantic information provided by the simulation. The complete system is evaluated in simulation through an autonomous mission including take-off, exploration, mapping, object detection, and navigation to a goal location. Experimental results demonstrate the functionality of the integrated system, while the documentation details the system architecture and operational principles and discusses current limitations. The source code is available on GitHub[1].**

I. Introduction

The deployment of autonomous UAVs in subterranean environments represents one of the most demanding frontiers in modern robotics. Unlike open-air operations, cave systems are characterized by a total lack of Global Navigation Satellite System (GNSS) signals, irregular geometries, and highly constrained lighting. The motivation for mastering these environments is rooted in critical real-world applications: search-and-rescue (SAR) operations in collapsed mines, structural inspection of hazardous tunnels, and the exploration of extraterrestrial lava tubes. In such scenarios, the ability of a robotic agent to navigate without a prior map while identifying specific points of interest (POIs) is not merely an advantage, but a necessity.

The primary objectives of this project are twofold. First, the system must achieve autonomous exploration of an unknown cave volume to locate four specific objects, designated as lanterns, while operating under strict time constraints. Second, the UAV is tasked with the simultaneous generation of a high-fidelity spatial representation, specifically a 3D voxel-grid or mesh-based map, which serves both for collision avoidance and as a permanent record of the subterranean structure.

We are provided the following:

- A Unity [1] simulation environment.
- A ROS-Simulation-Bridge providing ROS interfaces (e.g. topics, services).
- A package containing messages `nav_msgs` for communicating with rotary wing Micro Aerial Vehicles (MAVs).
- A package messages specific to MAV planning, especially polynomial planning `mav_planning_msgs` [2].

Regarding the scope and limitations, this research is conducted within a high-fidelity Unity-based simulation environment [1]. While simulation abstracts away certain physical "real-world" nuisances such as aerodynamic turbulence (ground effect) or hardware-specific sensor degradation, it provides a rigorous "Software-in-the-loop" (SITL) platform for validating complex algorithmic pipelines. The system is built entirely upon the ROS2 [3] framework, ensuring a modular architecture where perception, planning, and control modules communicate via standardized interfaces. This modularity ensures that the logic developed here can be ported to physical MAV platforms, with minimal modification.

The remainder of this paper is structured to provide a comprehensive overview of the drone's development and performance. We begin in Section II by outlining the project roadmap, specifically detailing the capability requirements and the distribution of work packages. Section III delves into the multi-layered system architecture, explaining the integration of high-level strategy and mission supervision of the state machine with the core modules for perception, path planning, and quadrotor control. The resulting simulation and visualization data are analyzed in Section IV, followed by a critical assessment of the technical limitations encountered during development in Section V. Finally, Section VI summarizes the project's key insights and proposes potential directions for future research.

II. Project Plan and Development Workflow

This section summarizes how the project requirements were translated into a structured development plan, how implementation was organized across work packages, and how contributions are documented for transparent traceability.

A. Requirements and Planning Basis

The project plan was derived from the course-level requirements and translated into subsystem-level implementation goals. The required mission capabilities are: ROS2-based modular execution, take-off and flight to the cave entrance,

[1] https://github.com/lkitzbichler/Autonomous-Systems-Sub-Terrain-Challenge-GR04

autonomous cave exploration, voxel-map generation, lantern detection and localization, and a safe return-and-land sequence. To keep implementation and evaluation traceable, each capability was assigned to a dedicated work package and mapped to a concrete ROS2 package/node interface.

During integration, one mission-level condition became explicit in the orchestration logic: return-home is triggered only after the complete lantern target is reached. In the current implementation, this corresponds to five tracked lanterns (including the reference lantern close to the start position).

B. Work Packages and Timeline

Development was organized into the following work packages:

1) Repo setup and tooling
2) Controller
3) Trajectory to cave entrance
4) State machine (mission orchestration)
5) Octomap and voxel mapping
6) Lantern detection
7) Path planning (exploration)
8) Testing and optimization
9) Report writing

The planned timeline and submission milestone are shown in Fig. 1. The plan was used as a baseline for implementation order, integration checkpoints, and final validation scheduling.

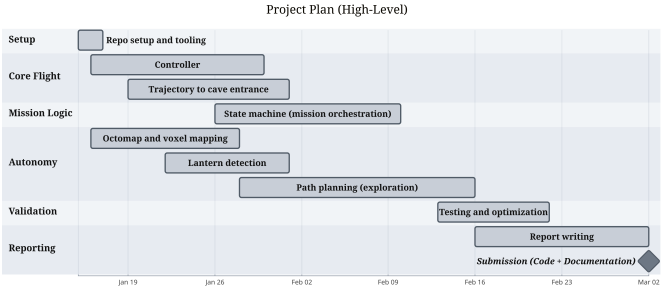

Fig. 1. Planned project timeline.

C. Development Workflow and Engineering Rules

The implementation followed an interface-first, package-oriented workflow. Each component was developed in isolation, then integrated via stable ROS2 topic contracts and heartbeat-based supervision. Reproducibility was treated as a mandatory quality criterion: dependency installation and build are provided via `build.bash`, and full-system startup is executed through one top-level launch entry (`run.bash`). While the underlying methodology and system architecture were developed entirely by the authors, Large Language Models (LLMs) [4], [5] were utilized as supportive tools for code implementation and technical documentation especially concerning the state machine and the path planner.

The codebase follows consistent engineering rules:

- ROS2 Jazzy with `ament_cmake`-based packages
- C++ node implementations with dedicated header/source separation
- Launch files and YAML parameter files per package
- Consistent topic/node naming and frame handling
- Traceable event/log output for integration and debugging

D. Software Structure and Repository Organization

The repository is organized as a modular ROS2 workspace (`ros2_ws/src`) with clear functional separation:

- **Supervision:** `statemachine_pkg` manages mission states, command dispatch, heartbeat monitoring, and mission-level transitions.
- **Control:** `controller_pkg` tracks sampled trajectories and publishes actuator commands.
- **Planning:** `basic_waypoint_pkg` handles entrance/takeoff/landing trajectories; `path_planning_pkg` performs autonomous exploration and return-home planning.
- **Perception and Mapping:** `lantern_detector_pkg` estimates lantern poses; `mapping_pkg` integrates the octomap server.
- **Simulation Interface:** `simulation` bridges Unity sensor/actuator streams and provides state-estimate corruption for robustness testing.

This structure enforces conceptual alignment between software modules and system functionality while keeping interfaces testable and replaceable.

E. Team Contribution by Work Package

To ensure transparent traceability of individual work, Table I - IX maps each work package to the responsible contributors, their relative share, and their concrete technical activities. The percentages are intended to reflect effective implementation effort per package and therefore sum to 100% in each column. It should be noted that different efforts were required for the work packages.

TABLE I
TEAM CONTRIBUTIONS: REPO SETUP AND TOOLING

Contributor	Share	Technical contribution
Kitzbichler, L	100%	Structured ROS2 workspace/package layout; integrated simulator into a single top-level launch chain; implemented one-command build/start scripts (`build.bash`, `run.bash`); adapted launch files and maintained setup/dependency documentation; set up a repository and a project on Github.

TABLE II
TEAM CONTRIBUTIONS: CONTROLLER

Contributor	Share	Technical contribution
Thimm, D Kitzbichler, L Waeger, S	33.3% 33.3% 33.3%	Integrated the core controller from the previous submission by wrapping it into a dedicated `controller_node`. Implemented the ROS2 integration, ensuring seamless communication between `command/trajectory`, `current_state_est`, and `rotor_speed_cmds`. Developed mission-critical command gating (`START`/`HOLD`/`LAND`/`ABORT`) and heartbeat-based supervision logic. Finalized the module by tuning control parameters and validating closed-loop behavior within the high-fidelity simulation to ensure robust interfacing with the overall system stack.

TABLE III
TEAM CONTRIBUTIONS: TRAJECTORY TO CAVE ENTRANCE

Contributor	Share	Technical contribution
Thimm, D Kitzbichler, L	40% 40%	Adapted and integrated the package into the system stack. Configured waypoint-based trajectory generation and implemented command-driven execution for `TAKEOFF`, `START`, and `LAND` sequences. Defined YAML waypoints and verified precise entrance positioning through simulation testing to ensure seamless interoperability with the mission control and the trajectory generation package.
Waeger, S	20%	Imported the trajectory-to-entrance implementation from the previous submission; supported testing and validating.

TABLE IV
TEAM CONTRIBUTIONS: STATE MACHINE / MISSION ORCHESTRATION

Contributor	Share	Technical contribution
Kitzbichler, L Thimm, D	44% 44%	Developed and implemented the core mission-state transitions and orchestration logic, covering the sequence `WAITING`→`TAKEOFF`→`TRAVELLING`→ `EXPLORING`→`RETURN_HOME`→ `LAND`→`DONE`. Implemented a targeted command dispatch system with optional target poses and integrated heartbeat-based supervision, including boot-timeout handling. Designed and implemented mission-level triggers (e.g. `detected_lantern_count` $\geq$ 5), specifically the transition to `RETURN_HOME` upon reaching the target threshold of five detected lanterns.
Heller, L Waeger, S Kristandra, B	4% 4% 4%	Contributed to initial state-machine structure and concept definition (team design session).

TABLE V
TEAM CONTRIBUTIONS: OCTOMAP AND VOXEL MAPPING

Contributor	Share	Technical contribution
Heller, L	100%	Integrated `octomap_server` in a dedicated launch; remapped point cloud input (e.g. `/realsense/depth/points`); configured map parameters (resolution/range/frames); generated and provided `octomap_binary` / voxel map representation for planner and state machine; ensured point cloud production/publishing and project integration.

TABLE VI
TEAM CONTRIBUTIONS: LANTERN DETECTION

Contributor	Share	Technical contribution
Heller, L	100%	Implemented synchronized semantic+depth processing; projected detections to 3D via depth-ray backprojection and transformed poses to `world`; merged repeated sightings using distance-threshold association and averaging; published lantern poses, counts and visualization markers.

TABLE VII
TEAM CONTRIBUTIONS: PATH PLANNING / EXPLORATION

Contributor	Share	Technical contribution
Kitzbichler, L Thimm, D	47% 47%	Implemented and integrated the autonomous exploration planner, conducting several design iterations including an initial OMPL-based [6] approach. Developed a ray-based path generation logic as the core planning component, ensuring both frontier exploration and obstacle avoidance within subterranean geometries. Established mission-critical backtracking, gating and robustness measures, validated through extensive simulation testing.
Kristandra, B	5%	Conducted additional research on OMPL [6] and planning approaches.
Waeger, S	1%	Participated in initial concept discussion.

TABLE VIII
TEAM CONTRIBUTIONS: TESTING AND OPTIMIZATION

Contributor	Share	Technical contribution
Thimm, D Kitzbichler, L Heller, L	33.3% 33.3% 33.3%	Executed comprehensive functional testing of all individual ROS2 packages followed by end-to-end integration tests using the top-level launch chain. Conducted cross-package parameter tuning across the controller, planner, and detector modules via YAML configurations. Performed systematic code reviews and detailed error analysis to stabilize mission behavior, specifically validating robustness and analyzing log/plot data. Final stabilization and cleanup. Set up new Ubuntu for testing twice.

TABLE IX
TEAM CONTRIBUTIONS: REPORT WRITING

Contributor	Share	Technical contribution
Thimm, D	32%	Set up latex project and structure. Authored report sections project plan and development workflow, system architecture parts, data export and visualization and limitations and discussion; Consolidated and polished full report and aligned architecture/workflow description with implemented ROS2 interfaces.
Heller, L	23%	Authored parts of system architecture like mapping/detection-related report sections; refined content and experimental description.
Kristandra, B	20%	Authored report introduction, quadrotor model and controller part.
Waeger, S	15%	Authored parts of System Architecture, Conclusion; Polished the report
Kitzbichler, L	10%	Authored section limitations and discussion. Consolidated and polished the full report; aligned architecture/workflow description with actual package interfaces; prepared figures/captions and contribution structure.

III. SYSTEM ARCHITECTURE

The architecture is partitioned into four primary functional layers: simulation abstraction, perception and mapping, motion planning, and mission supervision. Communication between these modules is handled via asynchronous topic publication and synchronous service requests, ensuring a decoupled and extensible software stack.

A. Simulation Interface and Data Acquisition

The interface between the ROS 2 [3] ecosystem and the Unity-based simulation environment is managed by the *simulation* package, which internally uses a dedicated *tcp_image* library to address the high bandwidth requirements of visual data. This library bypasses standard ROS transport mechanisms in favor of raw (TCP) sockets for the streaming of RGB, depth, and fisheye camera frames. Furthermore, a state corruptor node is integrated to inject Gaussian noise into the simulated ground-truth data, enabling the evaluation of control robustness under realistic uncertainties. This module was provided and not changed, except for fixing some coordinate frame missmatches between the semantic camera, depth camera and body frame.

B. Strategy and System Planning

The system is implemented as a modular ROS2 (Jazzy) [3] workspace in which each package has a clearly separated responsibility and communicates through explicit topic contracts. The runtime is started by running `run.bash`, which runs a top-level launch entry (`simulation.launch.py`), which instantiates supervision, perception, mapping, planning, control, and simulation-bridge components.

The architecture follows a state-machine-centered design that coordinates most nodes and is visualized as a rosgraph in figure 2.

The mission logic is implemented in `statemachine_pkg` via the `state_machine_node` which governs mission

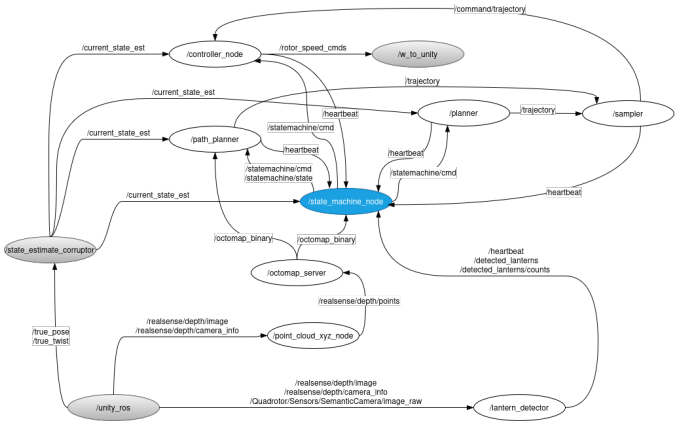

Fig. 2. Simplified ROS graph

states (e.g. takeoff, travelling, return-home, land), dispatches commands and monitors component health via heartbeat messages. A shared supervisory interface coordinates modules: `statemachine/cmd` distributes commands to planning and control nodes (with target filtering), `statemachine/state` provides global state awareness, and `heartbeat` messages enable runtime supervision. Subtask completion events are communicated via explicit information codes.

State estimation forms the basis of the control loop: Simulated true states are processed by `state_estimate_corruptor` to generate `/current_state_est`, which is consumed by supervision, planning, and control modules.

Command-driven flight phases are handled by the waypoint-based `planner` in `basic_waypoint_pkg` [7], which generates trajectories inputs for further sampling. Autonomous exploration and homing are realized in `path_planning_pkg`, where a graph-based `path_planner` performs exploration, backtracking, and return-home planning. Trajectories generated by either planner are converted into polynomial trajectories as executable control references by the `sampler` of `mav_trajectory_generation` [2], producing `MultiDOFJointTrajectory` commands.

Trajectory tracking is implemented in `controller_pkg` [8] (`controller_node`) which computes motor commands (`rotor_speed_cmds`) based on the commanded trajectory and the current state estimate.

Perception and mapping operate concurrently with the planning pipeline. Mapping integration is encapsulated in `mapping_pkg`, which launches and configures the external `octomap_server`. Depth data are transformed into point clouds and forwarded to the mapping server, which generates a probabilistic 3D occupancy representation (`octomap_binary` [9]) used by higher-level planning modules.

Object-level perception is implemented in `lantern_detector_pkg`, where synchronized semantic and depth streams are fused to estimate 3D object positions

in the world frame.

A dedicated `simulation` package provides a Unity–ROS interface for closed-loop testing, including sensor/actuator exchange. Shared message definitions in `mav_msgs` and `mav_planning_msgs` ensure modular interoperability across subsystems.

Visualization topics expose mission checkpoints, executed flight paths, trajectories, detected objects, and planning graph structures, enabling transparent runtime inspection.

C. Supervision and Low-Level Control

Mission supervision and low-level actuation are implemented as a layered ROS2 control chain. The supervisory layer is the `state_machine_node` (`statemachine_pkg`), which executes a finite-state mission logic and sends targeted commands on a shared command bus. The low-level layer (trajectory sampling and controller) converts high-level trajectories into actuator commands for the simulator.

1) Mission Supervision (`statemachine_pkg`): The state machine publishes `statemachine/state` and `statemachine/cmd`, and subscribes to `heartbeat`, `detected_lanterns`, `detected_lanterns/counts`, `current_state_est`, and `octomap_binary`. It additionally publishes visualization topics (`statemachine/viz/checkpoint`, `statemachine/viz/path_flight`).

Commands are sent via `statemachine/cmd` with an explicit `target` field. Each receiver executes only commands addressed to its own node name, enabling deterministic receiver-side filtering on a shared command bus.

The periodic supervision loop (`onTimer`) executes, in order:

i) state publication,
ii) heartbeat checks,
iii) checkpoint reach checks,
iv) transition/event handling,
v) retry logic for `TAKEOFF` and `LAND`,
vi) visualization publication.

Heartbeat-based liveness supervision is configured via `nodes` and covers `planner`, `sampler`, `path_planner`, `controller_node` and `lantern_detector`. The first five nodes publish periodic `statemachine_pkg/msg/Answer` messages (on `heartbeat`, typically at 1 Hz) with `RUNNING` status. A node is considered alive if the time since its last heartbeat is below `alive_tol_sec` (10 s). `octomap_server` is handled as a special case: since it does not publish the custom heartbeat, its liveness is inferred from ROS graph presence.

2) Implemented State Logic: The implemented mission states are `WAITING`, `TAKEOFF`, `TRAVELLING`, `EXPLORING`, `RETURN_HOME`, `LAND`, `DONE`, `ERROR`, and `ABORTED`. The effective transition chain is shown in Fig. 3.

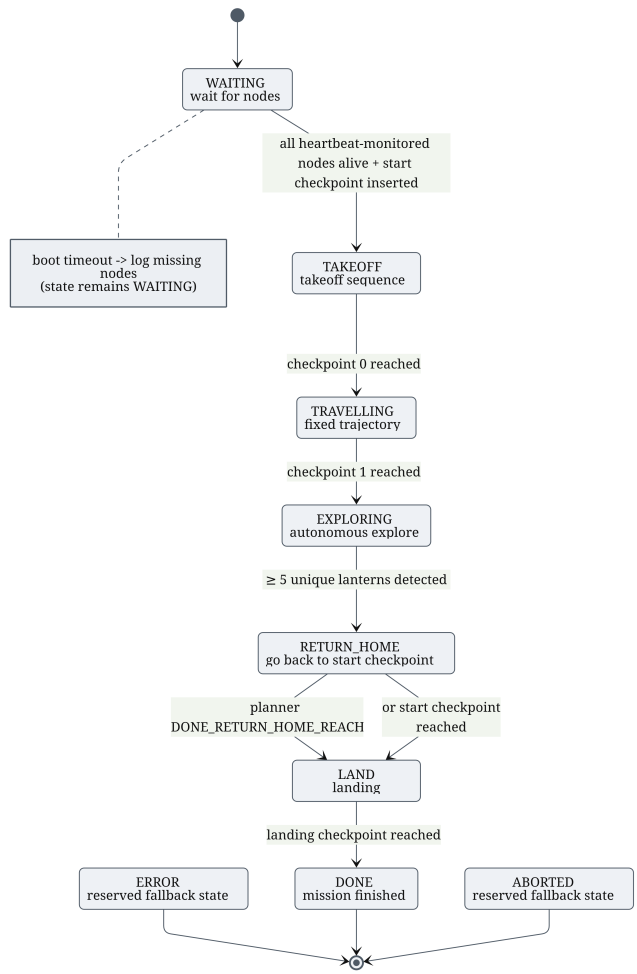

Fig. 3. Mission state machine implemented in `statemachine_pkg`.

To define the initial mission transitions, the state machine inserts a dynamic takeoff-reference checkpoint as soon as valid odometry is available:

$$\boldsymbol{p}_{\text{start-takeoff}} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 + 5.0\ \text{m} \end{pmatrix}$$

where $x_0,\ y_0,\ z_0$ denote the current UAV position components from `current_state_est` at the time the start checkpoint is inserted. This point is prepended to the active checkpoint list, i.e., after insertion $p_{\text{start-takeoff}}$ becomes checkpoint index 0. Its successful insertion is a precondition for the transition `WAITING` $\rightarrow$ `TAKEOFF`. Once this checkpoint (index 0) is reached during `TAKEOFF`, the state machine transitions to `TRAVELLING`.

Checkpoint reach criterion (used for state transitions): Checkpoint reach is evaluated by a 3-D Euclidean distance check. A checkpoint with index i is considered reached when

$$\left\|\boldsymbol{p}_{\mathrm{UAV}} - \boldsymbol{p}_{\mathrm{cp},i}\right\|_2 \leq d_{\mathrm{reach}}, \tag{1}$$

where $\boldsymbol{p}_{\mathrm{UAV}} \in \mathbb{R}^3$ is the current UAV position, $\boldsymbol{p}_{\mathrm{cp},i} \in \mathbb{R}^3$ is the position of checkpoint i, and $d_{\mathrm{reach}} =$ `checkpoints.reach_dist_m (YAML-Param)` (currently $0.5\,\mathrm{m}$).

Transition conditions in code:

- `WAITING` → `TAKEOFF`: all required heartbeat-publishing nodes are alive and the start checkpoint has been inserted.
- `TAKEOFF` → `TRAVELLING`: checkpoint index 0 reached according to the checkpoint-reach criterion in Equation 1.
- `TRAVELLING` → `EXPLORING`: checkpoint index 1 reached, using the same checkpoint-reach criterion (Equation 1). Checkpoint index 1 corresponds to the predefined static waypoint from `checkpoints.positions (YAML-Param)` $(-320.0,\ 9.0,\ 16.0)$, which represents the cave-entrance target in this setup.
- `EXPLORING` → `RETURN_HOME`: detected lantern count ≥ 5 (mission threshold in code).
- `RETURN_HOME` → `LAND`: either planner heartbeat reports `DONE_RETURN_HOME_REACHED` or the return-home target (start-takeoff checkpoint, index 0) is physically reached (same checkpoint-reach criterion).
- `LAND` → `DONE`: landing checkpoint reached.
- On entry to `DONE`, the state machine persists the final Octomap to **final_map.bt** if a valid Octomap has been received.
- `ERROR` and `ABORTED` are defined as reserved fallback states; there is no automatic incoming transition path in the current implementation.

On state entry, the state machine dispatches role-specific commands: controller `START/HOLD`, basic waypoint node (`planner`) `TAKEOFF/START/LAND`, and path planner `START/RETURN_HOME/HOLD`. `TAKEOFF` commands are periodically retried with period `takeoff_cmd_retry_sec` while the mission state remains `TAKEOFF`. In `LAND`, a landing target is generated from Octomap-based downward ray probing (with configurable hit criteria and adaptive XY radius), with optional fallback to the start position if map-based ground estimation fails.

Low-Level Control Execution: The controller runs a geometric tracking law looking into direction of movement and publishes `rotor_speed_cmds`. Mission-level gating is command-driven: `START` enables control output, while `HOLD/LAND/ABORT` disables control and publishes zero motor commands.

D. Quadrotor Model and Controller

The quadrotor dynamics is formulated in the configuration of special Euclidean group $\mathsf{SE}(3)$. We utilize a point-mass model with inertia from the rotors. We consider a quadrotor system equipped with four identical propellers positioned at the vertices of a square. Each propeller produces a thrust force and an aerodynamic torque acting exclusively along the normal to the vehicle's transverse plane [10].

We use the model proposed in [10] with some altercations to match the conventions used in ROS2. Fig. 4 illustrates the differences of coordinate systems used in the reference paper [10] and ROS convention. We define the inertial world frame W as $\{x_W^{ROS}, y_W^{ROS}, z_W^{ROS}\}$, and $\{b_1^{ROS}, b_2^{ROS}, b_3^{ROS}\}$ as the body-fixed frame B.

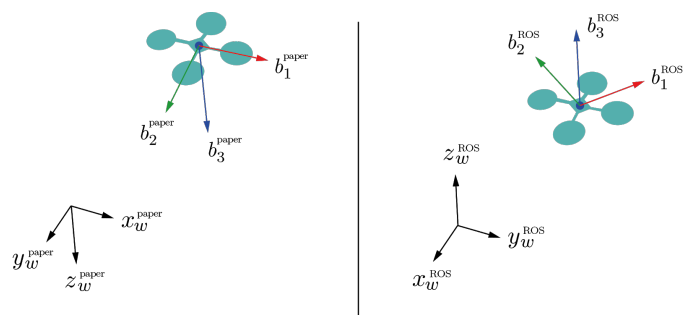

Fig. 4. Reference frame convention differences [8], [10]

The perception system relies on an integrated onboard camera suite, as illustrated in Fig. 5. This configuration features two lateral RGB cameras positioned with offset by $0.05\,\mathrm{m}$ from the longitudinal x_B axis, alongside a central sensor package that provides simultaneous depth and semantic data. While the RGB cameras capture standard chromatic visual information for localization, the depth sensor provides essential geometric data regarding the surrounding environment. Complementing these, the semantic camera performs pixel-level classification to identify objects within the scene. The perception system is mounted $0.1\,\mathrm{m}$ from the body frame origin.

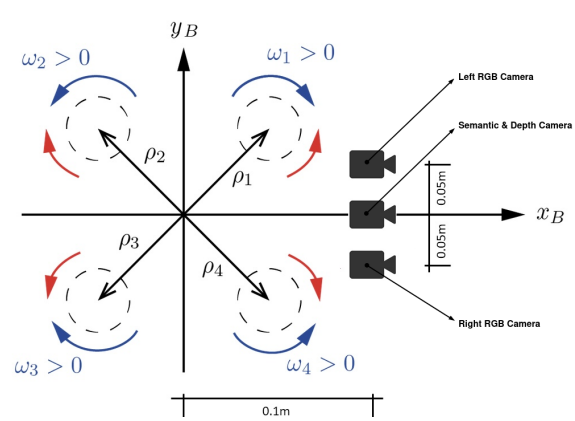

Fig. 5. Drone model conventions and onboard camera placement (adapted from [11])

The quadrotor's maneuverability is governed by the angular velocities ω_i of its four rotors, positioned at distances ρ_i from the center of mass. As illustrated in Fig. 5, the rotors operate in counter-rotating pairs to balance the reactive torque: rotors 1 and 4 rotate counter-clockwise ($\omega_1, \omega_4 > 0$), while rotors 2 and 3 rotate clockwise ($\omega_2, \omega_3 > 0$). This configuration allows for independent control of the aircraft's six degrees of freedom. By modulating the differential thrust between these pairs, the system can execute precise pitch, roll, and yaw maneuvers, while the net aerodynamic torque is canceled during steady-state hovering to maintain heading stability.

For this work, we are combining rigid body dynamics and modeling of aerodynamic effects. This serves as the basis model to design drone controller.

The equations of motion of the quadrotor [10] are given by:

$$\dot{\boldsymbol{p}}_B^W = \boldsymbol{v}_B^W \tag{2}$$

$$\begin{bmatrix} m\ddot{\boldsymbol{p}}_B^W \\ \boldsymbol{J}\dot{\boldsymbol{\omega}}_B^W \end{bmatrix} = -\begin{bmatrix} mg\boldsymbol{e}_3 \\ \boldsymbol{\omega}_B^W \times \boldsymbol{J}\boldsymbol{\omega}_B^W \end{bmatrix} + \begin{bmatrix} \boldsymbol{f}^W \\ \boldsymbol{\tau}^B \end{bmatrix} + \boldsymbol{J}_E^T \boldsymbol{\omega}_E^e \tag{3}$$

$$\text{where} \quad \boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$\boldsymbol{p}_B^W$ and $\boldsymbol{v}_B^W$ represent the location and the velocity of the center of mass of the drone in the body frame B with respect to the world frame W. m denotes the total mass of the drone and $\ddot{\boldsymbol{p}}_B^W$ is the acceleration of the center of mass of the drone. $\boldsymbol{J} \in \mathbb{R}^{3\times 3}$ as the inertia matrix with respect to the body-fixed frame, $\boldsymbol{\omega}_B^W$ and $\dot{\boldsymbol{\omega}}_B^W$ as the angular velocity and the angular acceleration respectively. We denote the vector $\begin{bmatrix} \boldsymbol{f}^W \\ \boldsymbol{\tau}^B \end{bmatrix}$ as the total input wrench and $\boldsymbol{J}_E^T \boldsymbol{\omega}_E^e$ as the interaction wrench mapped from the propeller to the drone body dynamics. Regarding actuation, $d \in \mathbb{R}$ represents the distance from the center of mass to each rotor center in the b_1^{ROS}, b_2^{ROS} plane.

The following equations are the wrench of a single propeller:

$$\boldsymbol{f}_i^B = \boldsymbol{R}_{S_i}^B \begin{bmatrix} 0 \\ 0 \\ c_f \end{bmatrix} \underbrace{w_i|w_i|}_{u_i}, \quad i = 1, \ldots, n \tag{4}$$

$$\boldsymbol{\tau}_i^B = \boldsymbol{R}_{S_i}^B \begin{bmatrix} 0 \\ 0 \\ \pm c_d \end{bmatrix} \underbrace{w_i|w_i|}_{u_i}, \quad i = 1, \ldots, n \tag{5}$$

where $\boldsymbol{f}_i^B$ is the force and $\boldsymbol{\tau}_i^B$ the torque of the propeller i with respect to the drone body $\boldsymbol{B}$. The matrix $\boldsymbol{R}_{S_i}^B$ denotes the rotation matrix of propeller i with respect to the drone body $\boldsymbol{B}$. We denote c_f as propeller lift coefficient, c_d as propeller drag coefficient.

The total input wrench can be described in the following equations [10]:

$$\boldsymbol{f}^W = \boldsymbol{R}_B^W \sum_{i=1}^{n} \boldsymbol{f}_i^B = \boldsymbol{R}_B^W \boldsymbol{F}_1 \begin{bmatrix} w_1|w_1| \\ \vdots \\ w_n|w_n| \end{bmatrix} = \boldsymbol{R}_B^W \boldsymbol{F}_1 \boldsymbol{u} \tag{6}$$

$$\boldsymbol{\tau}^B = \sum_{i=1}^{n} \boldsymbol{p}_{B,S_i}^B \times \boldsymbol{f}_i^B + \sum_{i=1}^{n} \boldsymbol{\tau}_i^B = \boldsymbol{F}_2 \begin{bmatrix} w_1|w_1| \\ \vdots \\ w_n|w_n| \end{bmatrix} = \boldsymbol{F}_2 \boldsymbol{u} \tag{7}$$

Combining the equations by substituting 6 and 7 into 3, we obtain the following:

$$\begin{bmatrix} m\ddot{\boldsymbol{p}}_B^W \\ \boldsymbol{J}\dot{\boldsymbol{\omega}}_B^W \end{bmatrix} = -\begin{bmatrix} mg\boldsymbol{e}_3 \\ \boldsymbol{\omega}_B^W \times \boldsymbol{J}\boldsymbol{\omega}_B^W \end{bmatrix} + \boldsymbol{J}_E^T \boldsymbol{\omega}_E^e + \begin{bmatrix} \boldsymbol{R}_B^W & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{F}_1 \\ \boldsymbol{F}_2 \end{bmatrix} \boldsymbol{u} \tag{8}$$

where

$$\boldsymbol{u} = \begin{bmatrix} w_1|w_1| \\ w_2|w_2| \\ w_3|w_3| \\ w_4|w_4| \end{bmatrix} \tag{9}$$

In this work, we assume that each propeller controls its generated force directly, neglecting the dynamics of the rotors and propellers. Furthermore, the thrust vector is assumed to be consistently aligned with the normal to the quadrotor's transverse plane (coplanar). Therefore, we can define:

$$\boldsymbol{F}_1 = c_f \begin{bmatrix} 0 & \ldots & 0 \\ 0 & \ldots & 0 \\ 1 & \ldots & 1 \end{bmatrix} = c_f \begin{bmatrix} \boldsymbol{0}^T \\ \boldsymbol{0}^T \\ \boldsymbol{1}^T \end{bmatrix} \tag{10}$$

We further assume that the torque produced by each propeller is proportional to its thrust magnitude through a constant drag coefficient c_d. As seen in Fig. 5, the first and the third propellers rotate clockwise, while the second and the fourth propellers rotate counterclockwise. Consequently, for a positive thrust f_i generated along the b_3 direction, the resulting reactive torque is modeled as $\tau_i = (-1)^{i-1} c_d f_i$. Therefore, we define:

$$\boldsymbol{F}_2 = \begin{bmatrix} c_f L_s & c_f L_s & -c_f L_s & -c_f L_s \\ -c_f L_c & c_f L_c & c_f L_c & -c_f L_c \\ c_d & -c_d & c_d & -c_d \end{bmatrix} \tag{11}$$

where $L_s = d\sin(\alpha)$ and $L_c = d\cos(\alpha)$ denote the effective moment arms relative to the body-fixed axes b_1^{ROS} and b_2^{ROS}. All of these assumptions are common [12], [13], and under these assumptions, the total thrust f in b_3 direction and the total moment M can be written as:

$$\begin{bmatrix} f_{b_3} \\ M_{b_1} \\ M_{b_2} \\ M_{b_3} \end{bmatrix} = \begin{bmatrix} \boldsymbol{F_1} \\ \boldsymbol{F_2} \end{bmatrix} \boldsymbol{u} \tag{12}$$

In this work, we utilized the geometric tracking controller proposed in [10]. Fig. 6 illustrates the controller structure. The translational motion of the quadrotor is primarily governed by the aggregate thrust vector $\boldsymbol{f}^W$. The direction of this

thrust remains persistently aligned with the third body-fixed axis b_3. For a given reference translational trajectory $\boldsymbol{p}_d(t)$, the stabilization of the translational dynamics is achieved by synthesizing the total thrust magnitude f and the desired orientation of the third body-fixed axis $b_{3,d}$.

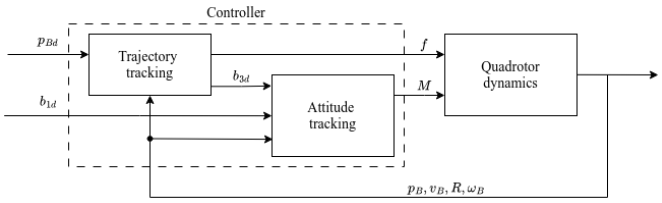

Fig. 6. Controller structure [10]

Once the desired direction of the third body-fixed axis b_{3_d} is established, the remaining degree of freedom in the desired attitude $\boldsymbol{R} \in SO(3)$ corresponds to the quadrotor's heading, where $\boldsymbol{R}$ is adopted as a simplified notation for $\boldsymbol{R}_B^W$ for brevity. This is determined by a reference axis $b_{1_d}(t)$, representing the desired pointing direction of the first body axis. Assuming b_{1_d} is not parallel to b_{3_d}, we define an orthonormal coordinate system by projecting b_{1_d} onto the plane normal to b_{3_d}, as illustrated in Fig. 7. Specifically, we compute:

$$b_{2_d} = \frac{b_{3_d} \times b_{1_d}}{\|b_{3_d} \times b_{1_d}\|}, \quad \boldsymbol{R}_d = [b_{2_d} \times b_{3_d}, b_{2_d}, b_{3_d}] \tag{13}$$

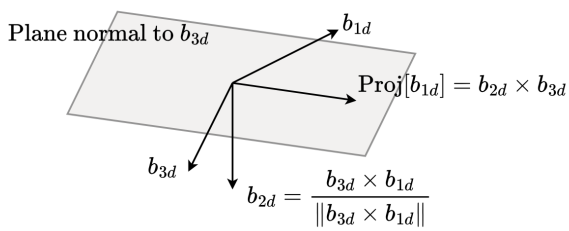

Fig. 7. Heading direction [10]

The control moment $\boldsymbol{M}$ is then designed to drive the quadrotor toward this desired orientation. In summary, a four-dimensional control input is designed to track a three-dimensional translational command and a one-dimensional heading direction. Specifically, the controller guarantees asymptotic convergence such that $\boldsymbol{p}(t) \rightarrow \boldsymbol{p}_d(t)$ and $\mathrm{Proj}[b_1(t)] \rightarrow \mathrm{Proj}[b_{1_d}(t)]$ as $t \rightarrow \infty$, where $\mathrm{Proj}[\cdot]$ denotes the normalized projection onto the plane orthogonal to b_{3_d}.

The tracking errors for the translational states (position and velocity) are defined as:

$$\boldsymbol{e}_p = \boldsymbol{p} - \boldsymbol{p}_d, \tag{14}$$

$$\boldsymbol{e}_v = \boldsymbol{v} - \boldsymbol{v}_d. \tag{15}$$

On the $SO(3)$ manifold, the attitude tracking error $\boldsymbol{e}_R$ is chosen as:

$$\boldsymbol{e}_R = \frac{1}{2}(\boldsymbol{R}_d^T \boldsymbol{R} - \boldsymbol{R}^T \boldsymbol{R}_d)^\vee, \tag{16}$$

where $(\cdot)^\vee$ denotes the vee map from $\mathfrak{so}(3)$ to $\mathbb{R}^3$, and R_d denotes the desired heading. Since $\dot{\boldsymbol{R}}$ and $\dot{\boldsymbol{R}}_d$ reside in different tangent spaces, the angular velocity tracking error is defined by transforming $\boldsymbol{\omega}_d$ into the current body frame:

$$\boldsymbol{e}_{\boldsymbol{\omega}_B} = \boldsymbol{\omega}_B \tag{17}$$

where we ignore the second term from [10] due to numerical differentiation of R_d and its negligible effects on closed-loop dynamics.

We further define the desired thrust direction b_{3_d} as:

$$b_{3_d} = -\frac{-k_{\boldsymbol{p}}\boldsymbol{e}_{\boldsymbol{p}} - k_{\boldsymbol{v}}\boldsymbol{e}_{\boldsymbol{v}} + mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d}{\| - k_{\boldsymbol{p}}\boldsymbol{e}_{\boldsymbol{p}} - k_{\boldsymbol{v}}\boldsymbol{e}_{\boldsymbol{v}} + mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d|} \tag{18}$$

with the assumption that:

$$\| - k_{\boldsymbol{p}}\boldsymbol{e}_{\boldsymbol{p}} - k_{\boldsymbol{v}}\boldsymbol{e}_{\boldsymbol{v}} + mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d\| \neq 0. \tag{19}$$

For a positive constant B, the desired trajectory satisfies:

$$\| - mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d\| < B. \tag{20}$$

Finally, the control inputs for thrust $\boldsymbol{f}$ and moment $\boldsymbol{M}$ are synthesized as:

$$\boldsymbol{f} = -(-k_x\boldsymbol{e}_{\boldsymbol{p}} - k_v\boldsymbol{e}_{\boldsymbol{v}} - mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d) \cdot \boldsymbol{R}\boldsymbol{e}_3, \tag{21}$$

$$\boldsymbol{M} = -k_R\boldsymbol{e}_{\boldsymbol{R}} - k_{\boldsymbol{\omega}}\boldsymbol{e}_{\boldsymbol{\omega}_B} + \boldsymbol{\omega}_B \times J\boldsymbol{\omega}_B. \tag{22}$$

where we ignore all the terms involving $\boldsymbol{\omega}_d$ and its time derivative as they are of the second order and have negligible effects on the closed-loop dynamics.

Beyond spatial coordinates, the system implements a dynamic orientation strategy to optimize the UAV's perception capabilities. A critical component of this logic is the calculation of a "velocity-aligned" desired yaw. When the vehicle's horizontal velocity exceeds a predefined threshold, the controller automatically computes a new target heading using the arc-tangent of the velocity components in the XY-plane. This ensures that the UAV's onboard sensors and cameras are always oriented toward the direction of travel. Such a behavior is essential for subterranean exploration, as it maximizes the visual "look-ahead" distance, allowing the perception pipeline to detect obstacles or lanterns well before the vehicle enters a potential collision volume.

E. Perception and Spatial Mapping

Environmental modeling is performed through a voxel-based representation. The `mapping_pkg` utilizes the public library Octomap [9] to convert raw point cloud data from simulated depth sensors into a 3D probabilistic occupancy grid. The conversion between the simulated depth image and the point cloud is performed by the public `depth_image_proc` [14] package provided by ROS2 [3]. The result of this can be seen in Fig. 8

The generated voxel grid (Fig. 9) serves as the fundamental data structure for collision checking and path planning.

Fig. 8. Point cloud generated from a single depth image.

Simultaneously, the `lantern_detector_pkg` processes visual data to localize targets. Targets within the semantic camera view are characterized by non-zero pixel intensities. Given the terrain topology and target sparsity, it is assumed that seperate targets do not overlap within the camera field of view.

The detection pipeline utilizes an approximate time synchronization policy to align frames from the semantic camera with depth telemetry. Because the sensor streams operate at different frequencies, this policy ensures temporal alignment within a defined window. For each frame, a binary mask $\boldsymbol{M}$ on the semantic image is generated via thresholding.

For each valid pixel (coordinates u, v) in $\boldsymbol{M}$, depth values d are sampled from the depth image if they reside within the stable operating range $[d_{min}, d_{max}]$. The 3D position vector in the camera frame, $\boldsymbol{P}_{cam}$, is recovered by calculating the arithmetic mean of the back-projected coordinates for all n valid pixels:

$$\boldsymbol{P}_{cam} = \frac{1}{n}\sum_{i=1}^{n}\begin{bmatrix}(u_i - c_x)\frac{d_i}{f_x}\\(v_i - c_y)\frac{d_i}{f_y}\\d_i\end{bmatrix} \tag{23}$$

where the intrinsic parameters (f_x, f_y) and the principal point (c_x, c_y) are extracted from the camera projection matrix $\boldsymbol{P}$. The resulting vector is transformed into the global world frame using the Transform Library (TF2) to produce the instantaneous measurement $\boldsymbol{P}_{meas}$.

Temporal consistency is maintained by clustering detections based on a Euclidean distance threshold ϵ. If a new detection $\boldsymbol{P}_{meas}$ is within ϵ of an existing target $\boldsymbol{P}_{t,old}$, the target position is updated recursively using a moving average:

$$\boldsymbol{P}_{t,new} = \frac{(\boldsymbol{P}_{t,old} \cdot N) + \boldsymbol{P}_{meas}}{N+1} \tag{24}$$

where N represents the cumulative detection count. This recursive filter mitigates transient sensor noise and improves the structural integrity of the localized target map.

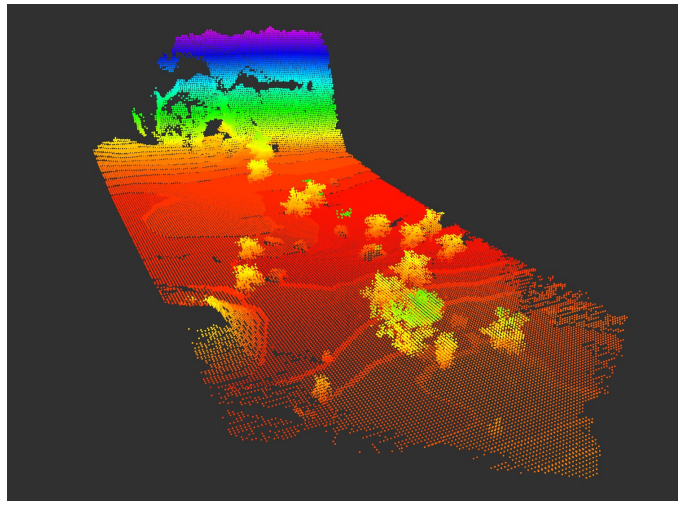

Fig. 9. Voxelmap of the environment up to the cave entrance with voxels colored by height.

F. Path Planning and Exploration

The `path_planning_pkg` is implemented as the ROS2 node `path_planner`. It performs autonomous cave exploration by maintaining a persistent topological graph, generating local collision-aware motion targets, detecting loop closures, and executing route-based backtracking to stored branch targets (hole markers) as well as command-driven return-home routing. For parts of the implementation Large Language Models (LLMs) [4], [5] have been used.

1) Node Interfaces and Operating Modes: The node subscribes to `statemachine/cmd`, `statemachine/state`, `current_state_est`, and `octomap_binary`. It publishes `trajectory`, `path_planner/current_plan`, `path_planner/graph_markers`, `path_planner/candidate_markers`, and `heartbeat`. Supported internal modes are `IDLE`, `TRANSIT_RECORD`, `EXPLORE`, `BACKTRACK`, `RETURN_HOME`, `HOLD`, and `ABORTED`. Mission commands `START`, `SWITCH_TO_EXPLORE`, `RETURN_HOME`, `HOLD`, and `ABORT` are handled explicitly. `RETURN_HOME` is not self-triggered by the planner; it is executed only when commanded by the state machine.

2) Input Validity and Output Gating: Motion output (`trajectory`, `path_planner/current_plan`) is enabled only in active planning modes (`EXPLORE`, `BACKTRACK`, `RETURN_HOME`) and only with valid odometry and map input. Debug publications (graph/candidate markers) are still updated by the timer. Odometry must be fresh and frame-consistent. Map validity requires available binary octomap. If `map_timeout_sec (YAML-Param)` ≤ 0, map-age checking is disabled, but map availability and frame consistency are still mandatory. This prevents publishing motion commands when odometry or map inputs are invalid or stale.

3) Map Query and Safety Primitives: Collision and traversability checks are implemented by `isOccupied`, `isUnknown`, `isFree`, `raycast`, and `clearance`. `isOccupied` includes occupancy inflation radius `inflation_m`. `isFree` is:

$$\begin{aligned}\mathrm{isFree}(\boldsymbol{p}) = \neg\mathrm{isUnknown}(\boldsymbol{p}) \;\wedge\; \neg\mathrm{isOccupied}(\boldsymbol{p}) \\ \wedge\; \mathrm{clearance}(\boldsymbol{p}) \geq c_{\min}.\end{aligned} \tag{25}$$

where $\boldsymbol{p} \in \mathbb{R}^3$ is a queried point and $c_{\min}$ is `min_clearance_m (YAML-Param)`.

`raycast` samples along a segment and returns the first blocking sample. These primitives are used consistently for candidate classification, line-of-sight checks for graph edges, side-opening probing, and local path validation.

4) Transit Recording and Persistent Graph: Transit recording is activated only when `statemachine/state == TRAVELLING`. In `TRANSIT_RECORD`, transit nodes are recorded with spacing `transit_node_spacing_m (YAML-Param)` and connected sequentially, creating the initial backbone of the global graph. The first valid transit node initializes the home reference.

The planner maintains an undirected graph

$$G = (V, E), \tag{26}$$

where V denotes the set of graph nodes and E denotes the set of undirected connections (edges) between these nodes. Each node stores position, status, timestamp, transit flag, and observation count, and each edge stores metric length and cost.

Node insertion uses merge-or-create logic. If a node $v \in V$ exists within merge radius, its position is updated by weighted averaging:

$$\boldsymbol{p}_v \leftarrow \frac{n_v\,\boldsymbol{p}_v + \boldsymbol{p}_{\mathrm{new}}}{n_v + 1}, \quad n_v \leftarrow n_v + 1, \tag{27}$$

where $\boldsymbol{p}_v \in \mathbb{R}^3$ is the previous node position, $\boldsymbol{p}_{\mathrm{new}} \in \mathbb{R}^3$ is the new sample, and n_v is the previous observation count. Otherwise, a new graph node is created. Global routing uses Dijkstra [15] shortest-path search on valid graph edges. This provides a mission-persistent memory of explored structure and enables route-based recovery.

5) Local Candidate Fan and Deterministic Selection: The candidate points $\boldsymbol{p}_{i,j}$ are generated within a spherical fan defined by the horizontal and vertical half-fields-of-view α_ψ and α_θ, using N_ψ and N_θ bins at a fixed radial distance d_c: Candidate points are

$$\boldsymbol{p}_{i,j} = \boldsymbol{p}_t + d_c \begin{bmatrix} \cos\theta_j \cos(\psi_t + \phi_i) \\ \cos\theta_j \sin(\psi_t + \phi_i) \\ \sin\theta_j \end{bmatrix}, \tag{28}$$

where $\boldsymbol{p}_t \in \mathbb{R}^3$ is the current UAV position, $\phi_i \in [-\alpha_\psi, \alpha_\psi]$, and $\theta_j \in [-\alpha_\theta, \alpha_\theta]$. Each pair (ϕ_i, θ_j) defines one ray direction in the local fan. The ray starts at $\boldsymbol{p}_t$ and its endpoint $\boldsymbol{p}_{i,j}$ is used as the candidate probe point for map queries. Thus, the planner evaluates a fixed discrete set of local motion directions per cycle in a reproducible way. With default parameters ($N_\psi = 7$, $N_\theta = 3$, $\alpha_\psi = 60°$, $\alpha_\theta = 30°$), the fan has 21 rays. Both ray count and angular spread are parameterized; thus, 21 rays is a default operating point, not a fixed limit.

Each candidate is classified as `FREE`, `BLOCKED`, or `UNKNOWN`. Known-path rejection suppresses candidates near already traversed transit/visited regions.

Selection is deterministic:

$$J_{\mathrm{ang}} = |\phi| + 0.7|\theta|, \tag{29}$$

where ϕ and θ are candidate relative yaw and pitch (degrees). Candidates are prioritized by class (`prefer_unknown_over_free`), then by J_{ang}, then by fixed index order. The deterministic ordering avoids oscillations and makes behavior reproducible in logs. Figure 10 illustrates one planning cycle of the local yaw–pitch fan. The selected ray is highlighted, while rejected rays remain visible, showing the deterministic class-and-angle based candidate ranking in a single snapshot. White segments are sampled rays; colored points show candidate classes and selected target. Yellow graph nodes are persisted hole markers used for backtracking.

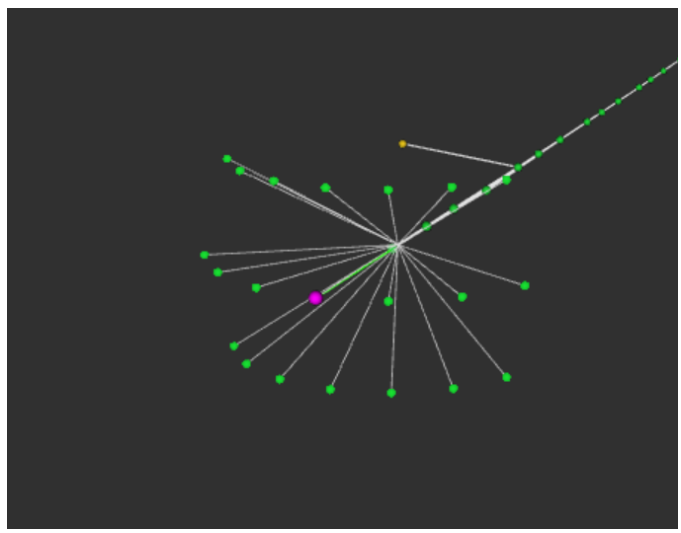

Fig. 10. Local candidate fan in RViz2

6) Side-Opening Detection and Hole Marker Storage: Branch detection uses side bundles around $\pm 90°$ relative yaw, with additional pitch sampling and long probe distance. For N side rays with open depths ρ_k, the pass fraction is

$$\eta = \frac{1}{N} \sum_{k=1}^{N} \mathbf{1}(\rho_k \geq \rho_{\min}) \tag{30}$$

where $\rho_{\min}$ is the minimal expected distance to have no collision `side_opening_min_depth_m (YAML-Param)`.

A side opening is accepted by strict or weak condition:

$$\eta \geq \eta_{\min} \quad \text{or} \quad (\eta \geq 0.85\,\eta_{\min}) \wedge (\rho_{\mathrm{best}} \geq \rho_{\mathrm{weak}}), \tag{31}$$

where $\eta_{\min}$ is side_opening_required_fraction (YAML-Param),

$$\rho_{\text{best}} = \max_{k=1,\dots,N} \rho_k, \tag{32}$$

and

$$\rho_{\text{weak}} = \max(0.1, \rho_{\min} - \max(1.0, 0.1\rho_{\min})) . \tag{33}$$

Streak confirmation over side_opening_confirm_cycles (YAML-Param) is required, and marker creation occurs on the rising edge of stable detection. Hole placement distance is

$$d_{\text{hole}} = \min(d_{\max}, \max(0.5, \rho_{\text{best}} - m_{\text{safety}})) , \tag{34}$$

where $d_{\max} =$ candidate_distance_m (YAML-Param) and m_{safety} is derived from min_clearance_m (YAML-Param). If occupied, the target is pulled back iteratively along the same direction until a non-occupied point is found (checked with isOccupied).

Figure 11 shows a confirmed side-opening event and the resulting hole-marker placement in RViz. The yellow marker denotes the stored branch target, while the green nodes represent the already traversed main route. After placement, this marker is handled as a deferred backtracking target via the pending/queued hole-node logic.

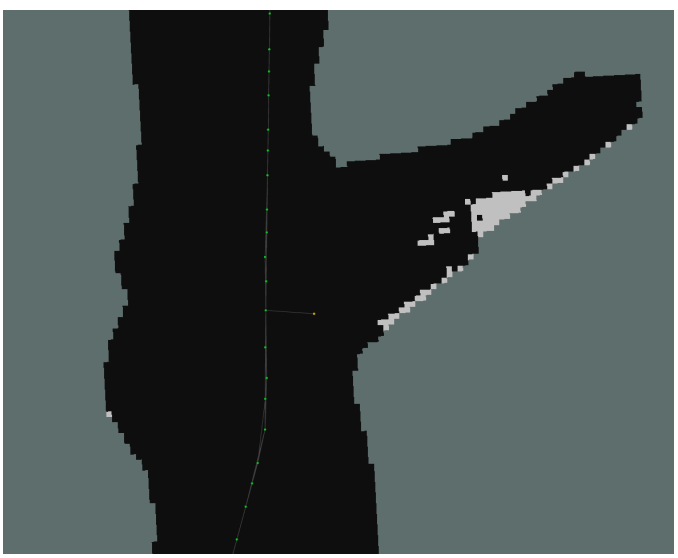

Fig. 11. Detected side-opening and resulting hole-marker placement (yellow graph nodes) in RViz (path_planner/candidate_markers).

Stored side nodes are assigned high frontier score and managed persistently via:

- pending_hole_node_id (active target),
- queued_hole_node_ids (additional targets).

Queue normalization removes invalid/duplicate entries and promotes the next valid pending target. Holes are stored as deferred high-priority targets to be revisited via routing.

7) Graph Update and Loop-Closure Logic: Each cycle, the graph anchor is updated at current UAV position, connected to the previous anchor, and additionally linked to nearby old/visited nodes if line-of-sight is occupancy-safe (no occupied cells along the segment and no occupied endpoint in the planner checks).

Loop closure uses two evidence classes:

- **Strong evidence:** an anchor revisit or a newly added edge reaches old known structure without an equivalent preexisting route.
- **Soft evidence:** a preexisting route to an old node satisfies

$$(h \geq h_{\min}) \ \vee \ (L \geq L_{\min}), \tag{35}$$

for consecutive cycles of the same soft-loop candidate node (streak confirmation), where h is route hop count, L is route length, $h_{\min} =$ loop_closure_soft_min_hops (YAML-Param), and $L_{\min} =$ loop_closure_soft_min_route_len_m (YAML-Param).

The implementation additionally uses guards (minimum node age, recent-anchor suppression, and overlap checks) to reduce false positives. On confirmed closure in EXPLORE, the planner sets force_graph_replan_after_loop_ and reports EVENT_LOOP_CLOSED. Mode switching is intentionally decoupled from loop-detection itself.

8) Decision Flow in EXPLORE, BACKTRACK, and RETURN_HOME: In EXPLORE, a confirmed loop closure sets force_graph_replan_after_loop_. If this flag is active and a reachable hole route exists, the planner starts that route and switches to BACKTRACK. If no hole route is currently feasible, it remains in EXPLORE and retries in subsequent cycles.

Normal explore-goal priority is:

i) selected local fan candidate,
ii) nearest graph FRONTIER/UNVISITED node.

High-score side-hole frontiers are excluded as direct EXPLORE goals and reserved for route-based backtracking.

If no local explore goal is available but unvisited frontier nodes exist, the planner can switch to BACKTRACK by routing to the nearest reachable unvisited frontier. The mode switch occurs only if route initialization to the selected frontier succeeds.

In BACKTRACK state, the planner prioritizes pending or queued hole routes. If no such route is reachable, it falls back to reachable unvisited frontiers. The planner first attempts to generate a route to the current pending hole target. If this target is unreachable, queued hole targets are evaluated sequentially after queue normalization and promotion. At any given time, only one hole target is active (pending), while additional targets remain in the queue. Route progress advances when

$$\left\| \boldsymbol{p}_t - \boldsymbol{p}_{r,k} \right\|_2 \leq d_{\text{reach}}, \tag{36}$$

where $\boldsymbol{p}_t$ is current UAV position, $\boldsymbol{p}_{r,k}$ is current route node, and $d_{\text{reach}} = \max(0.2,$ route_node_reached_dist_m$)$.

After reaching a pending hole target, pending is cleared, queue is normalized/promoted, and mode returns to `EXPLORE`.

`RETURN_HOME` is command-driven by mission logic (state machine). In this project, mission logic transitions to return-home when the lantern counter reaches 5. Home-node resolution in the planner is:

i) existing valid home node,
ii) explicit return target from command,
iii) fallback to last transit node.

After route completion, planner publishes `DONE_RETURN_HOME_REACHED` and switches to `HOLD`. Figures 12 and 13 show the 2-D cave projection and the complete exploration path generated by the planner. Revisited junction regions and overlapping route segments indicate loop-closure handling and subsequent graph-based replanning/backtracking.

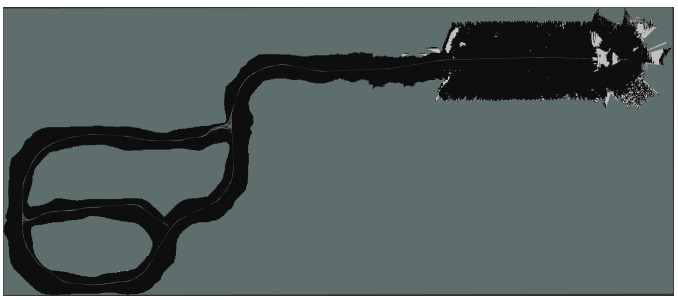

Fig. 12. 2D projection of the explored cave with the complete mission path.

Fig. 13. Zoomed 2D view of the exploration path at a junction-rich region.

9) Local Path and Trajectory Output: Given goal g, local path generation is:

- direct segment $\boldsymbol{p}_t \to \boldsymbol{g}$, if line-of-sight is occupancy-safe;
- otherwise one-hop bridge via fan candidate $\boldsymbol{p}_m$, minimizing

$$J_{\text{bridge}} = \|\boldsymbol{p}_t - \boldsymbol{p}_m\| + \|\boldsymbol{p}_m - \boldsymbol{g}\| + 0.1|\phi_m| + 0.05|\theta_m|. \quad (37)$$

where $\boldsymbol{p}_t$ is current position, $\boldsymbol{p}_m$ is bridge point, $\boldsymbol{g}$ is goal, and ϕ_m, θ_m are candidate relative yaw/pitch (degrees). Both subsegments must satisfy occupancy-safe visibility (i.e., no occupied cells on the segment and no occupied endpoint in the planner checks).

The waypoint path is converted to piecewise-linear `PolynomialTrajectory4D`. For segment k between waypoints $\boldsymbol{p}_k$ and $\boldsymbol{p}_{k+1}$, segment time is

$$\Delta t_k = \max\left(\Delta t_{\min}, \frac{\|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\|}{v_{\text{nom}}}\right), \quad (38)$$

where v_{nom} = `trajectory.nominal_speed_mps` (`YAML-Param`) and $\Delta t_{\min}$ = `trajectory.min_segment_time_sec` (`YAML-Param`).

IV. Data Export and Visualization

During simulation, the system autonomously records the **Voxel Map** and **Lantern** data. Once the drone reaches its designated goal position, the final voxel map is serialized and saved for post-processing and analysis.

A. Voxel Occupancy Maps

The occupancy data is stored in the binary tree format (`.bt`). To visualize the generated environment, you may use Octovis, a standalone visualization tool. Assuming it is installed, it can be launched using:

```
octovis ros2_ws/final_map.bt
```

The final reconstructed occupancy map is shown in Figure 14 and Figure 15. Together, these two views show the full map and how the cave corridors are connected.

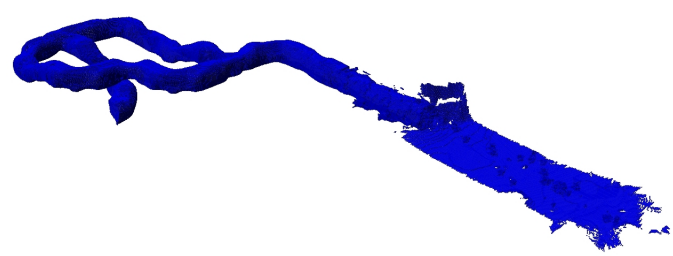

Fig. 14. Final voxel occupancy map visualized in Octovis.

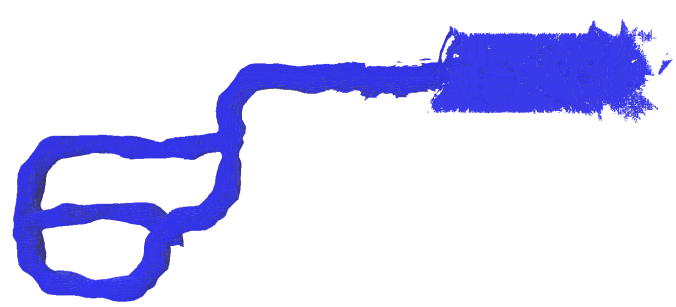

Fig. 15. Final voxel occupancy map in top-view projection, highlighting the overall cave layout.

B. Lantern and Telemetry Data

The global spatial coordinates of each lantern as well as the cumulative number of frames, each lantern was within the sensor's field of view are exported to a `.csv` file. The final estimated lantern coordinates obtained from the logged dataset are summarized in Table X.

TABLE X
RESULT: LANTERN POSITIONS.

Lantern-ID	X	Y	Z
0	-59.35	0.95	6.52
1	-598.37	-9.18	4.83
2	-743.41	-230.44	2.38
3	-1044.32	-157.81	-36.37
4	-800.53	-282.29	-76.79

V. LIMITATIONS AND DISCUSSION

- **Project constraints: late start and ROS2 ecosystem maturity.** The project started comparatively late, and the codebase had only recently transitioned to ROS2. As a consequence, the available off-the-shelf tooling and community-maintained packages for our specific requirements (notably 3D planning and tight integration into a ROS2 mission stack) were more limited than in ROS1. This affected initial development velocity and increased the implementation effort for integration and debugging.
- **Planning approach iteration: OMPL [6] integration and tuning effort.** We initially attempted to use OMPL [6] for 3D path planning. While an OMPL-based pipeline could be integrated, we were not able to tune the planner configuration to reliably generate smooth, goal-directed motion in our scenario. In practice, erratic trajectories complicated the identification of specific parameter influences on planner behavior. This necessitated a systematic refinement to ensure stability and improve overall exploration performance. We therefore switched to an alternative planning approach that better matched the constraints and enabled stable mission execution.
- **Exploration logic failure mode: false "unvisited branch" hypotheses.** The exploration strategy relies on lateral ray casting to detect potential side corridors and branch points. If a ray does not register a collision within its sensing range, the system interprets this as evidence of an unexplored branch. However, the absence of a detected obstacle does not necessarily imply unexplored space — the vehicle may have previously traversed that direction, and the current sensing configuration simply does not capture this history. As a result, the planner can enqueue revisit goals that do not correspond to genuinely unexplored areas. This behaviour indicates that the branch-detection heuristic would benefit from additional context (e.g., incorporating the traversed path history into the branch test, or using occupancy/visibility constraints to suppress already-covered directions).
- **Simulation-driven testing overhead and parameter tuning.** System performance depends strongly on multiple parameter sets (controller, mapping, perception, and planning). However, iterative tuning is costly, as simulation runs are time-consuming and cannot be accelerated. This constrained systematic parameter search and regression testing within the project timeframe. Further performance gains are therefore likely achievable through extended tuning and automated evaluation.
- **Apparent flight "pitching" during direction changes.** During abrupt direction changes, the vehicle exhibits noticeable pitching transients, which may appear unstable despite the system remaining controllable and succesfully completing the mission. This behavior is likely caused by aggressive setpoint updates and/or controller gains that prioritize tracking performance over smoothness. Improved trajectory smoothing and refined controller tuning could mitigate these transients and enhance perceived flight quality.
- **Computational performance is essential.** The code being tested on multiple systems proved very sensitive behavior according computational performance due to real-time iterations. Potentially voxel map doesn't exist early enough to calculate take-off point.
- **Loop closure results show significant differences.** Loop closure is not very stable causing the UAV to need different amounts of fly-by connections to detect the loop closure.
- **Random results due to many different parameters.** Due to stochastic state estimation and varying computational side processes, the results of repeated runs can differ significantly. If the drone observes the environment from a different direction, its behavior may change due to forward-weighting effects in the planning or perception pipeline. Additionally, the voxel map can evolve differently depending on the viewing direction, which in turn influences subsequent perception and planning decisions. These feedback effects contribute to variability in overall system performance across runs.

VI. CONCLUSION

This paper presented the architecture and implementation of a modular ROS2-based autonomous aerial system designed for supervised exploration, semantic target detection, and safe return-to-home operation in a simulated environment. The system follows a strictly layered design, separating mission supervision, perception, mapping, planning, control, and simulation interfacing into clearly defined packages with explicit topic-level contracts. This modularization enables reproducibility, extensibility, and systematic verification of individual subsystems.

At the core of the architecture, a finite-state machine orchestrates high-level behavior through a deterministic command bus with receiver-side filtering. The supervisory logic ensures robust execution through heartbeat monitoring, checkpoint validation, and condition-driven state transitions. High-level plan-

ners generate polynomial trajectories for waypoint navigation and graph-based exploration, while a trajectory sampler and geometric $\mathsf{SE}(3)$-based controller translate these trajectories into actuator commands. The integration of semantic-depth-based lantern detection with Octomap-based environment modeling enables mission-relevant perception. Specifically the reliable localization and positioning of detected lanterns in the global world frame and also supports a reliable and map-consistent return to the designated landing location.

The proposed structure demonstrates how complex autonomous behavior can be realized through clean separation of concerns and well-defined communication interfaces. The layered control chain, combined with persistent planning structures and supervised lifecycle management, provides a scalable foundation for future extensions such as multi-agent coordination, real-world deployment on hardware platforms, or integration of advanced perception and optimization techniques.

At the same time, the development process highlighted the limited availability of mature and readily integrable ROS2 packages for fully autonomous 3D aerial exploration. In particular, autonomous exploration in the cave environment proved to be the most challenging subsystem due to sparse reference implementations and limited reusable software infrastructure. Given the constrained development timeframe and the overall system complexity, the realization of a fully functional UAV system was only achieved with support from AI-generated code [4], [5]. For this reason, the exploration package is the component with the greatest optimization potential, especially regarding efficiency, and structural refinement. Further improvements are described in Section V.

Overall, the system serves as a structured reference implementation for ROS2-based aerial autonomy, emphasizing architectural clarity, deterministic supervision, and modular extensibility, while clearly identifying exploration as the primary avenue for future improvement.

References

[1] M. Ryll and A. A. S. TUM, "Unity simulation." https://syncandshare.lrz.de/getlink/fi7Vw11aA5WwyMBRPVQYun/. [Accessed 03-02-2026].

[2] M. Achtelik, M. Burri, H. Oleynikova, R. Bähnemann, and M. Popović, "Autonomous systems 2025 - trajectory generation assignment repository." https://github.com/ethz-asl/mav_trajectory_generation, 2021. [Online; Accessed 01-03-2026].

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[4] OpenAI, "Chatgpt (version 5.2 thinking) [high-reasoning large language model]," 2026. Utilized for complex algorithmic logic synthesis and ROS2 architecture optimization. Accessed: March 3, 2026.

[5] OpenAI, "Codex (version 5.3) [high-reasoning ai-powered code generation model]," 2026. Employed for code refinement and C++ interface generation. Accessed: March 3, 2026.

[6] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, pp. 72–82, December 2012. https://ompl.kavrakilab.org.

[7] M. Ryll, "Autonomous systems 2025 - trajectory generation assignment repository." https://github.com/TUM-AAS/autonomous-systems-2025-trajectory-generation-AutSys_Labs_ROS2_TrajectoryGeneration, 2025. [Online; Accessed 01-03-2026].

[8] M. Ryll, "Autonomous systems 2025 - geometric tracking controller assignment repository." https://github.com/TUM-AAS/autonomous-systems-2025-geometric-tracking-controller-AutSys_Labs_ROS2_GeoTrackControl, 2025. [Online; Accessed 12-02-2026].

[9] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, 2013. Software available at http://octomap.github.com.

[10] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor uav on se(3)," in *49th IEEE Conference on Decision and Control (CDC)*, pp. 5420–5425, 2010.

[11] M. Ryll, "Autonomous systems 2025 - repository." https://gitlab.lrz.de/aas_public/autonomoussystems2025, 2025. [Online; Accessed 12-02-2026].

[12] P. Castillo, R. Lozano, and A. Dzul, "Stabilization of a mini-rotorcraft having four rotors," in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, vol. 3, pp. 2693–2698 vol.3, 2004.

[13] A. Tayebi and S. McGilvray, "Attitude stabilization of a vtol quadrotor aircraft," *IEEE Transactions on Control Systems Technology*, vol. 14, no. 3, pp. 562–571, 2006.

[14] "depth_image_proc - ROS Package Overview — index.ros.org." https://index.ros.org/p/depth_image_proc/. [Accessed 12-02-2026].

[15] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.